

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	KONGARA JOGESH	Reg. No.:	192525035
Date:	7/10/26	Duration:	60 minutes

Code of Conduct

I <u>Kongara Jogesh(192525035)</u> (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student:k.jogesh

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted one problem statement. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. Justify the Chosen Methodology/Framework

Assigned Problem Statement

19. Autonomous Incident Response and Root-Cause Analysis Platform

Industry Context

Complex software systems generate large volumes of logs, metrics, alerts, and deployment events. During incidents, engineers must correlate these signals quickly to identify root causes. Alert overload and fragmented observability can delay recovery and increase operational risk.

Problem Statement

Design and develop an Autonomous Incident Response and Root-Cause Analysis Platform that correlates monitoring data, deployment changes, service dependencies, and incident histories to identify probable causes and recommend safe remediation.

The platform should integrate application monitoring, centralized logging, incident management, infrastructure-as-code, and deployment systems. Automated remediation should operate within predefined safety boundaries and require human approval for high-impact changes.

Objectives

- Correlate metrics, logs, traces, and deployment events
- Detect and group related alerts
- Identify probable root causes
- Recommend appropriate remediation actions
- Automate low-risk recovery within predefined limits
- Maintain incident histories and audit trails
- Support human approval for high-impact actions

Expected Outcomes

- Reduce alert overload
- Improve root-cause identification
- Reduce incident response and recovery time
- Improve reliability of production services
- Reduce repetitive operational effort
- Provide auditable and controlled automated incident response

The problem statement supplied with the assessment is therefore treated as the primary case-study source for the analysis in this report.

Proposed System Architecture

The architecture below maps the problem statement's required integrations to a controlled incident-analysis and remediation pipeline.

Proposed Platform - High-Level Architecture

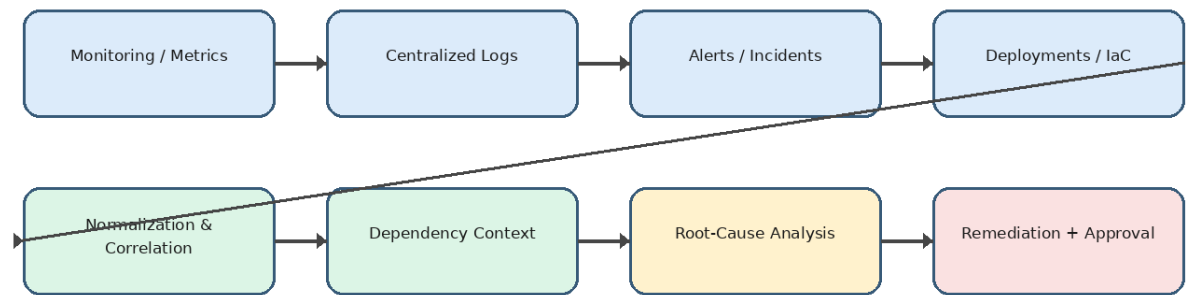


Figure 1. High-level architecture of the proposed platform.

Architecture Component Description

Component	Purpose	Main Data
Application Monitoring	Collect application health and performance signals.	Metrics, traces, health events
Centralized Logging	Aggregate and search operational logs.	Application and infrastructure logs
Alert / Incident Management	Receive alerts and maintain incident records.	Alerts, incidents, severity
Deployment / IaC Integration	Connect changes with incidents.	Deployment events, configuration changes
Normalization & Correlation	Standardize and connect events from multiple sources.	Correlated event stream
Dependency Context	Represent service and infrastructure relationships.	Service/dependency graph
Root-Cause Analysis	Rank probable causes from correlated evidence.	Cause candidates + evidence
Remediation & Approval	Recommend and control recovery actions.	Action plan, approval state
Audit & History	Store decisions and outcomes for learning.	Timeline, approvals, actions

Stakeholder Analysis

Stakeholder Interaction Model

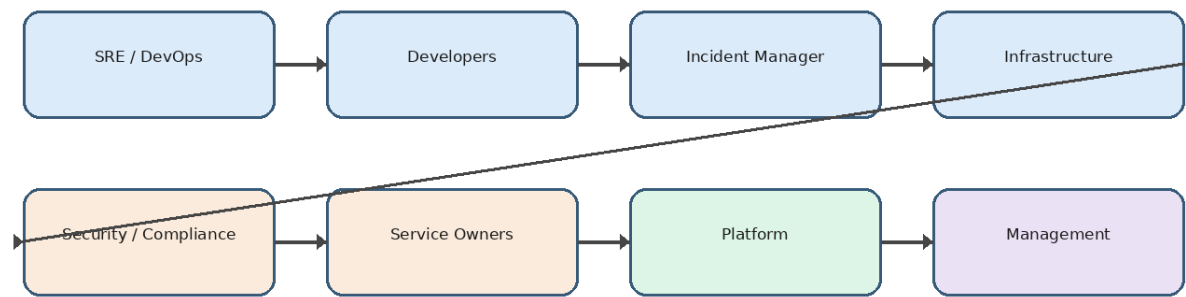


Figure 2. Stakeholder interaction model.

Stakeholder	Needs	Responsibilities	Key Challenge
SRE / DevOps	Fast diagnosis and safe recovery	Monitor, investigate, approve actions	Alert overload and time pressure
Developers	Application-level evidence	Analyze defects and service behavior	Finding relevant logs and changes
Incident Manager	Coordination and visibility	Manage incident timeline and communication	Many teams and fragmented information
Infrastructure Team	Stable infrastructure	Maintain systems, IaC and dependencies	Complex dependencies
Security / Compliance	Controlled automation	Access control, audit and governance	Prevent unsafe actions
Service Owners	Business impact visibility	Define criticality and approve risk	Balancing availability and business risk
Management	Reliability and cost outcomes	Governance and priorities	Measuring operational improvement

Stakeholder Challenges – Detailed Analysis

- Alert fatigue: engineers may receive many duplicate or related alerts during one incident.
- Context switching: responders move between monitoring, logging, deployment, incident, and infrastructure tools.
- Dependency complexity: a failure in one service can create symptoms in several dependent services.
- Automation risk: an incorrect action can make an incident worse, so safety boundaries and approval gates are essential.
- Knowledge loss: without structured incident histories, lessons from previous incidents may not be reused.
- Governance: high-impact changes require authorization, traceability, and accountability.

Current System / Existing Approach Analysis

The current approach described by the problem statement can be interpreted as a fragmented, largely manual incident-response process. Engineers correlate information from multiple operational systems and use experience to determine the likely root cause before performing recovery actions.

Aspect	Current / Manual Approach	Observed Gap	Proposed Improvement
Signal Collection	Data exists across multiple tools.	No unified context.	Centralized ingestion and normalization.
Alert Handling	Responders inspect alerts individually.	Duplicate / related alerts create overload.	Incident grouping and correlation.
Root Cause	Engineer manually compares evidence.	Diagnosis can be slow and experience-dependent.	Evidence-based root-cause ranking.
Dependencies	Often checked manually.	Hidden relationships delay diagnosis.	Dependency context / graph.
Remediation	Engineer chooses and executes action.	Repeated work and human error risk.	Controlled recommendations and automation.
Approval	May rely on operational procedures.	High-impact actions need explicit governance.	Human approval gate.
Learning	Incident reports stored separately.	Difficult to reuse historical knowledge.	Structured history and audit trail.

Strengths of the Current Approach

- Human judgment is available for complex or unusual failures.
- Existing operational tools can continue to provide valuable data.
- High-impact decisions can be reviewed by experienced engineers.
- Established recovery procedures can be applied where known.

Limitations / Gaps

- Slow cross-tool correlation.
- High alert volume and duplicate signals.
- Dependence on individual experience.
- Repeated manual recovery work.
- Limited automated learning from historical incidents.
- Risk of inconsistent actions during stressful incidents.

Proposed Software Solution

The platform should be implemented as modular services with controlled interfaces. Each module should have a clear responsibility so that the system can be maintained and scaled without changing the entire platform.

Module	Inputs	Processing	Outputs
Data Ingestion	Logs, metrics, traces, alerts, deployments	Collect and validate events	Normalized event stream
Normalization	Heterogeneous events	Map to common schema, remove duplicates	Standardized records
Incident Correlation	Alerts and events	Group related signals by time, service, dependency	Incident context
Dependency Analysis	Service and infrastructure metadata	Build relationships and impact paths	Affected-component context
Root-Cause Analysis	Incident context + history	Compare evidence and rank causes	Probable causes + confidence/evidence
Remediation Recommendation	Cause + service state	Select predefined actions	Recommended action plan
Approval / Automation	Action plan + policy	Check risk and authorization	Approved / rejected / executed action
Audit & History	All decisions and actions	Persist timeline and outcome	Auditable incident record
Dashboard	Incident records	Present status and evidence	Responder view / reports

Functional Requirements

- The system shall ingest monitoring, logging, alert, deployment, and incident data.
- The system shall normalize incoming events into a common representation.
- The system shall group related alerts into incidents.
- The system shall maintain service and infrastructure dependency context.
- The system shall identify and rank probable root causes.
- The system shall recommend safe remediation actions.
- The system shall execute only predefined low-risk actions automatically.
- The system shall require human approval for high-impact actions.
- The system shall maintain incident histories and audit trails.
- The system shall provide dashboards and reports for authorized users.

Non-Functional Requirements

Requirement	Target / Design Consideration
Reliability	No single operational component should unnecessarily prevent incident analysis.
Scalability	Architecture should support increasing event and service volumes.
Performance	Incident correlation and dashboard retrieval should be responsive during active incidents.
Security	Role-based access, least privilege, secure credentials, and controlled approvals.
Maintainability	Modular services, documentation, testing, and version control.
Usability	Clear incident summary, timeline, evidence, cause ranking, and action status.
Auditability	Record who/what/when/why for important decisions and actions.

Incident Response Workflow

Incident Response Workflow

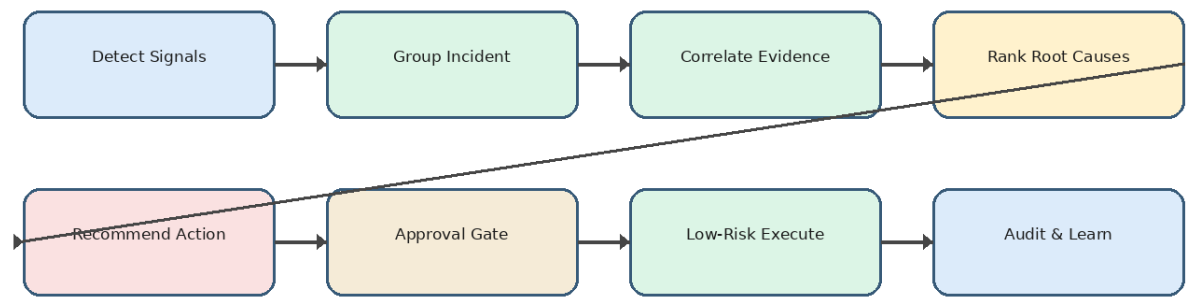


Figure 3. Controlled incident-response workflow.

Workflow Explanation

Step	Description
1. Detect Signals	Monitoring and operational tools generate metrics, logs, alerts, traces, and deployment events.
2. Group Incident	Related alerts are combined so responders work with an incident rather than many independent notifications.
3. Correlate Evidence	The platform aligns timing, services, deployments, dependencies, and historical information.
4. Rank Root Causes	Candidate causes are ranked using available evidence and their relationship to the incident.
5. Recommend Action	The system proposes a safe remediation plan and explains the supporting evidence.
6. Approval Gate	Low-risk actions can follow predefined automation rules; high-impact changes require human approval.
7. Execute	Approved or permitted low-risk actions are executed with safeguards.
8. Audit & Learn	The platform records the incident, action, result, and approval trail for future analysis.

Safety Boundaries for Autonomous Remediation

- Allow only pre-approved action types.
- Use role-based authorization and least privilege.
- Apply service criticality and blast-radius limits.
- Require human approval for high-impact changes.
- Provide dry-run / validation before risky actions where feasible.
- Record every automated action and its result.
- Provide rollback or recovery procedures for supported actions.

Software Engineering Principles

Principle	How It Is Applied	Benefit
Modularity	Separate ingestion, correlation, RCA, remediation, approval and audit services.	Independent change and testing.
Scalability	Scale event-processing components based on workload.	Supports growing systems.
Maintainability	Use clean interfaces, reusable code, documentation and tests.	Lower long-term maintenance effort.
Reliability	Health checks, retries, redundancy, safe fallback and controlled automation.	Higher service continuity.
Usability	Incident dashboard focuses on impact, evidence, probable cause and action.	Faster responder decisions.
Security	RBAC, least privilege, encryption, secrets protection and approvals.	Reduces unauthorized actions.
Auditability	Record evidence, decisions, approvals, actions and outcomes.	Supports accountability and learning.

5. Justify the Chosen Methodology / Framework

Selected approach: Agile with Scrum practices and DevOps integration.

The assessment requires selection and justification of a suitable SDLC model or software-development framework. Agile/Scrum is appropriate because the proposed platform contains multiple integrations and evolving operational requirements. Features can be delivered incrementally, tested with stakeholders, and improved from operational feedback. DevOps integration is appropriate because the system connects development, deployment, infrastructure, monitoring, and operations.

Agile / Scrum Element	Application to This Project
Product Backlog	Prioritize ingestion, correlation, RCA, dashboard, approval, and automation features.
Sprint Planning	Select a manageable set of platform capabilities for each iteration.
Daily Scrum	Track integration, development, testing, and operational blockers.
Sprint Review	Demonstrate features to SRE, developers, security, and service owners.
Retrospective	Improve development and incident-analysis processes based on feedback.
CI/CD	Continuously test and safely deliver platform changes.
DevOps Monitoring	Use operational feedback to improve reliability and performance.

Expected Outcomes, Risks and Future Enhancements

Expected Outcome	How the Proposed Platform Addresses It	Measurement Idea
Reduce alert overload	Group related alerts and suppress repetitive signals.	Alerts per incident / responder.
Improve root-cause identification	Correlate evidence and rank probable causes.	Diagnosis time and accuracy feedback.
Reduce response/recovery time	Automate correlation and safe low-risk actions.	MTTR / recovery duration.
Improve reliability	Use controlled remediation and dependency awareness.	Incident recurrence / service reliability.
Reduce repetitive effort	Automate approved routine recovery tasks.	Manual actions avoided.
Auditable response	Maintain complete incident and approval history.	Audit completeness.

Risks and Mitigation

Risk	Impact	Mitigation
Incorrect root-cause ranking	Wrong remediation recommendation.	Show evidence, confidence, and require review for high-impact actions.
Unsafe automation	Incident may worsen.	Predefined safety boundaries, least privilege, approval gates, rollback.
Poor data quality	Incorrect correlations.	Validation, normalization, monitoring of ingestion quality.
Integration complexity	Delayed development and maintenance.	Modular connectors and incremental integration.
Security exposure	Unauthorized access or actions.	RBAC, secure credentials, encryption, audit trails.
Low user trust	Engineers ignore recommendations.	Explainable evidence and feedback loops.

Future Enhancements

- Historical incident learning and improved cause ranking.
- Predictive incident detection before user-visible failure.
- More detailed dependency graphs and service-impact analysis.
- Simulation / dry-run environments for remediation.
- Progressive expansion of automation based on measured safety.
- Business-impact-aware prioritization and service-level objectives.

System Context and Use-Case Analysis

The following diagrams provide additional software-engineering views of the assigned platform. They translate the supplied problem statement into understandable models while keeping the analysis focused on incident correlation, root-cause identification, safe remediation, and human approval.

Proposed Platform - High-Level Architecture

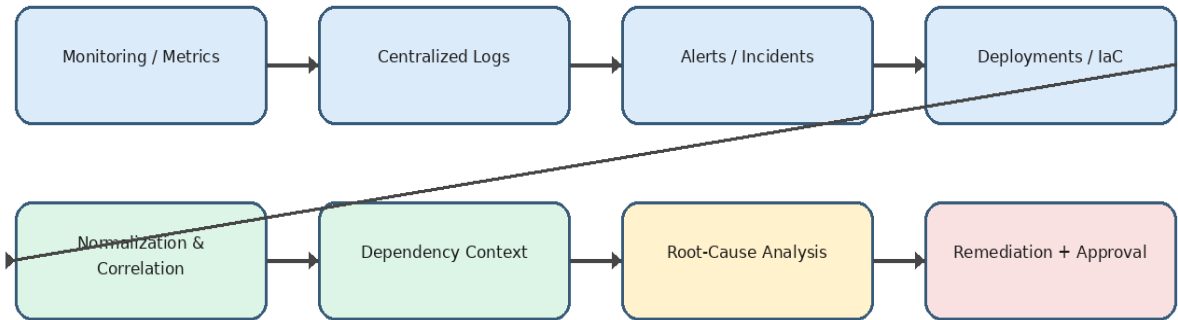


Figure 4. System context diagram.

Stakeholder Interaction Model

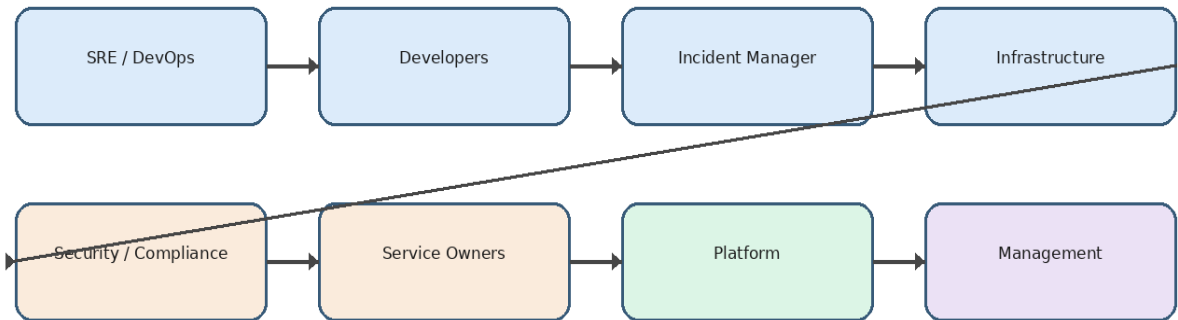


Figure 5. Use-case view.

Data Flow and Information Processing

A staged information flow reduces coupling between external systems and the decision-making components. The data is first collected and normalized, then correlated into an incident context, analyzed for probable causes, and finally passed through policy and approval controls.

Incident Response Workflow

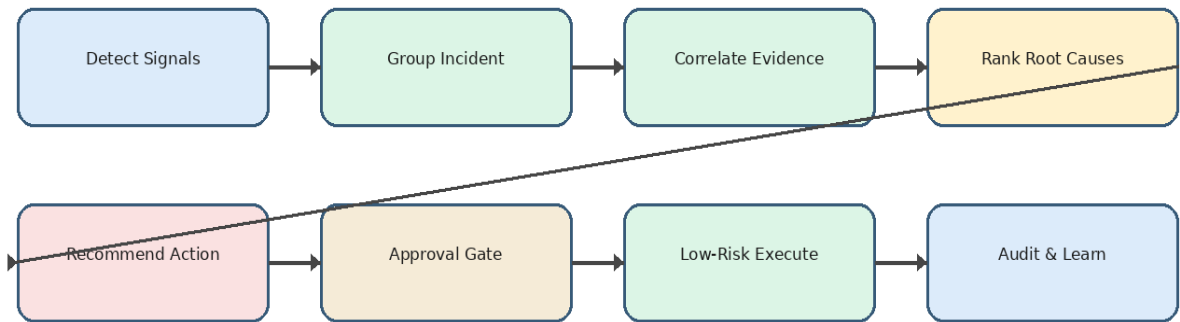


Figure 6. Level-1 data-flow diagram.

Stage	Input	Processing	Output
Collection	Logs, metrics, alerts, traces	Validate and timestamp	Raw event stream
Normalization	Heterogeneous events	Common schema, deduplication	Standardized events
Correlation	Standardized events	Time, service, dependency matching	Incident context
Analysis	Incident context	Rank candidate causes	Probable causes
Decision	Cause + policy	Risk and authorization check	Recommendation / approval
Action	Approved plan	Controlled execution	Recovery result
Learning	Timeline + outcome	Persist and analyze history	Audit / reference

Detailed Incident-Handling Sequence

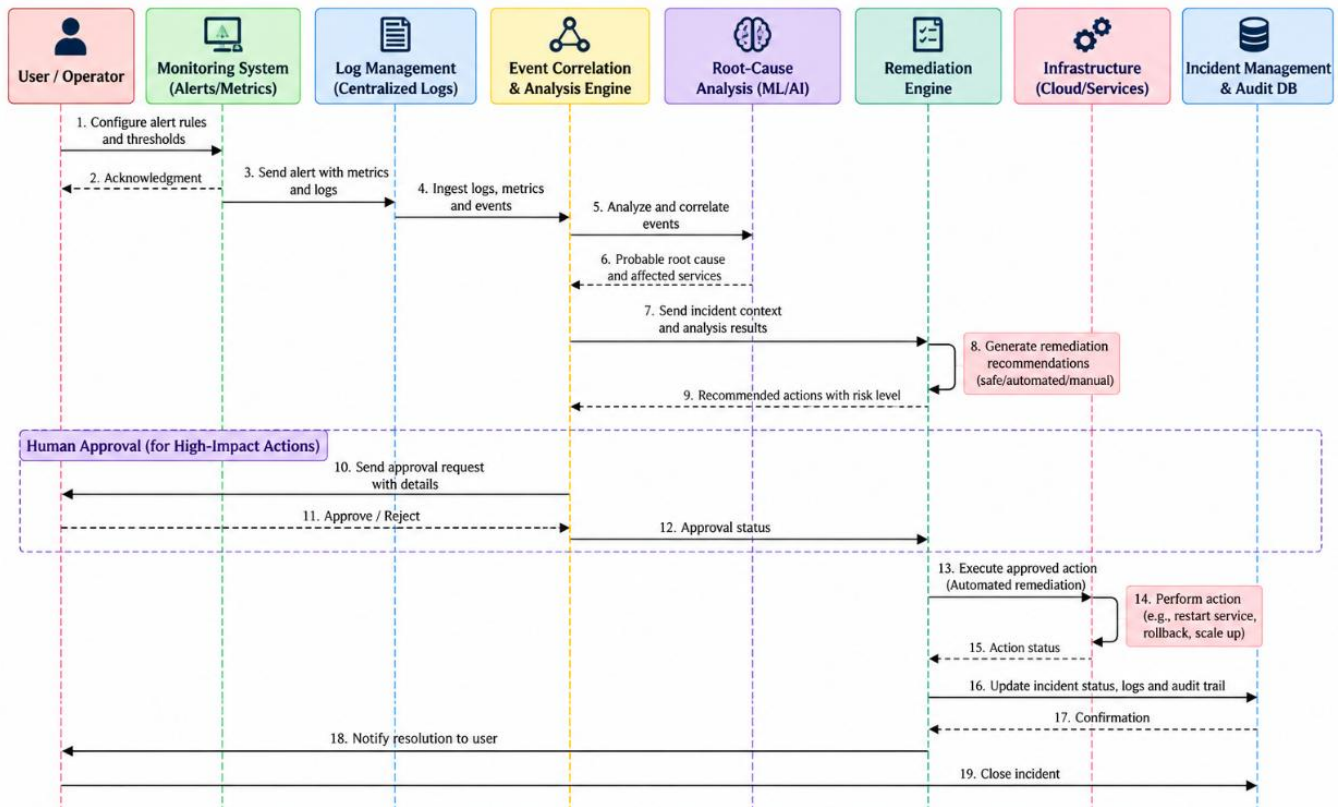


Figure 7. Sequence diagram for incident diagnosis and controlled remediation.

Design interpretation:

- Monitoring detects an abnormal signal and forwards it to the platform.
- The platform correlates the signal with related alerts, deployments, dependencies and incident history.
- The RCA component ranks candidate causes and preserves supporting evidence.
- Responders receive one incident context rather than manually switching between multiple tools.
- The recommended action is checked against risk and authorization policies.
- Approved actions are executed and their results are written to the audit trail.

Logical Data Model and Data Governance

Logical Data Model

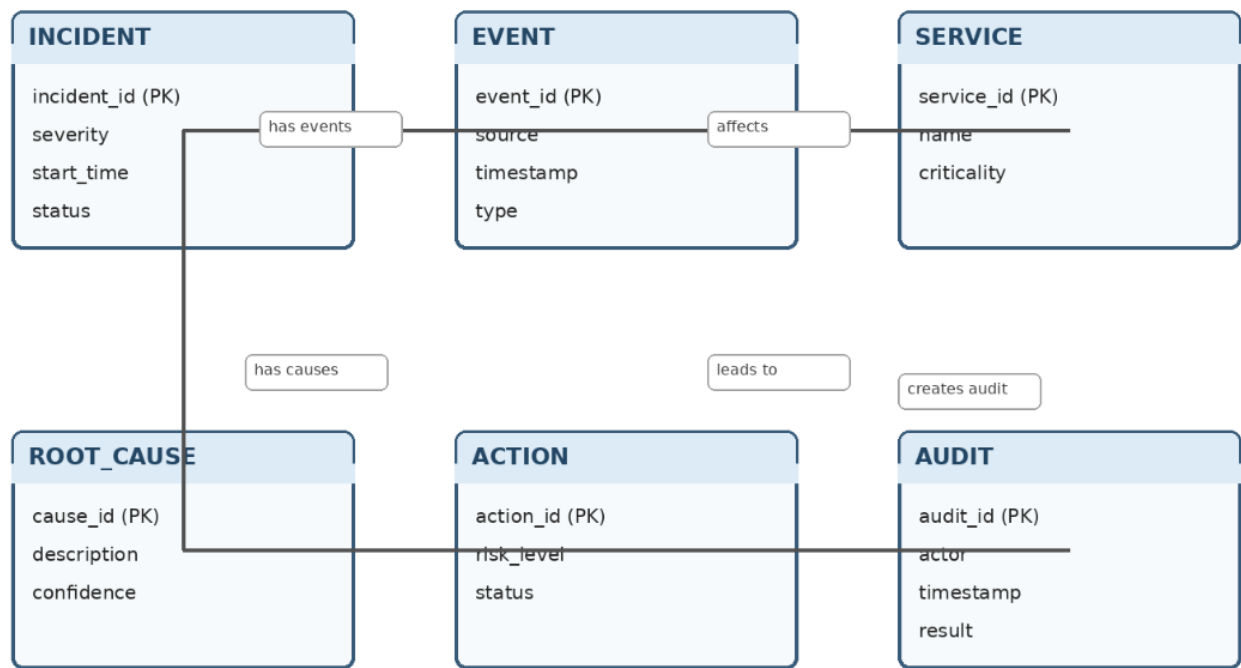


Figure 8. Logical data model.

Entity	Important Attributes	Purpose
Incident	ID, severity, start time, status	Represents the operational incident.
Event	ID, source, timestamp, type	Stores operational signals.
Service	ID, name, criticality	Provides service context.
Root Cause	ID, description, confidence	Stores candidate causes and evidence.
Action	ID, risk level, status	Represents remediation.
Audit	ID, actor, timestamp, result	Provides governance and accountability.

Data governance considerations:

- Use consistent identifiers and timestamps across integrations.
- Restrict sensitive operational data to authorized roles.
- Retain sufficient history for learning while following organizational retention policies.
- Record the source and time of important evidence.
- Protect audit records against unauthorized modification.

Deployment and Infrastructure View

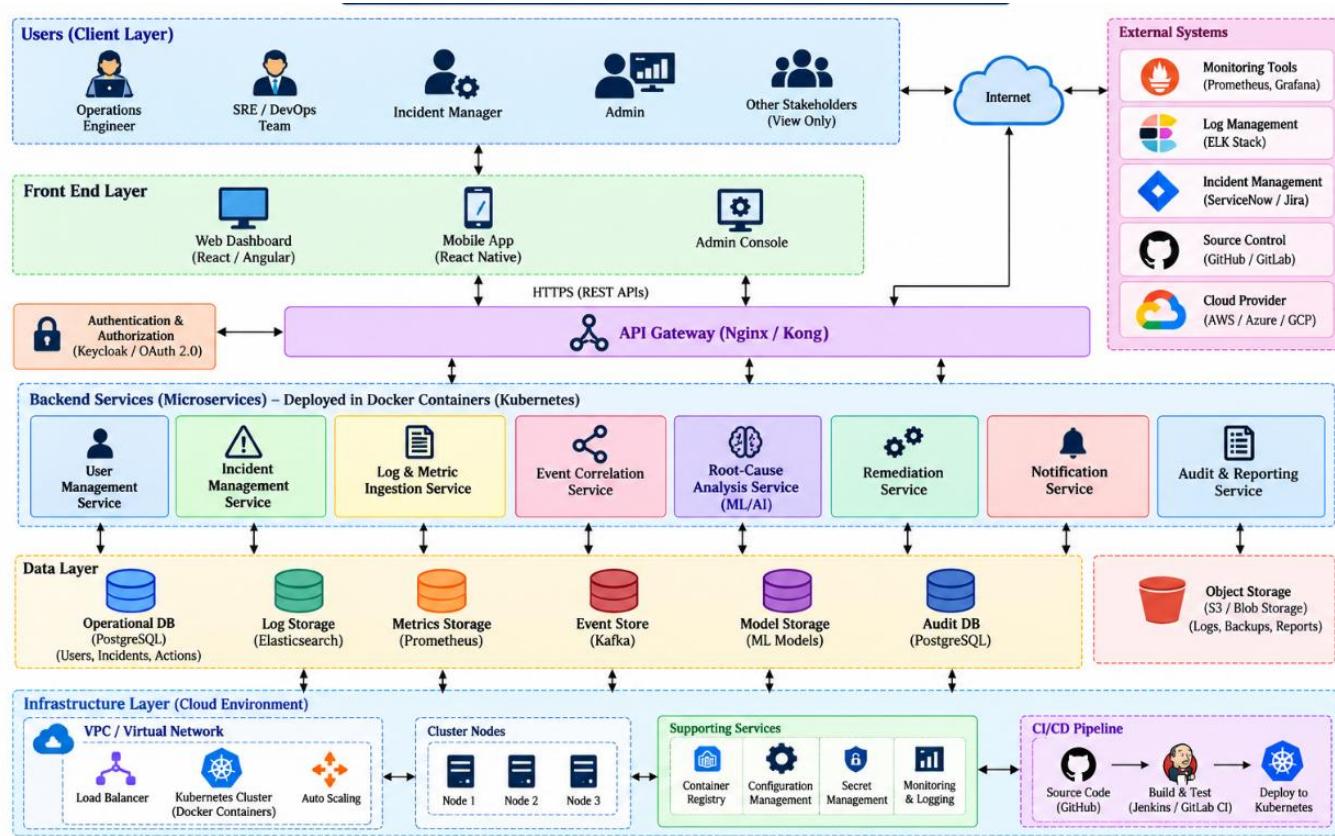


Figure 9. Illustrative deployment view.

Layer	Responsibility	Design Concern
User / API	Dashboard and operational interfaces	Authentication and usability
Analysis	Correlation and RCA services	Scalability and low latency
Data	Event, incident and audit stores	Durability and access control
Governance	Policy and approval services	Least privilege and authorization
Integration	Monitoring, logging, deployment connectors	Reliable interfaces and validation

Risk-Based Remediation and Safety Controls

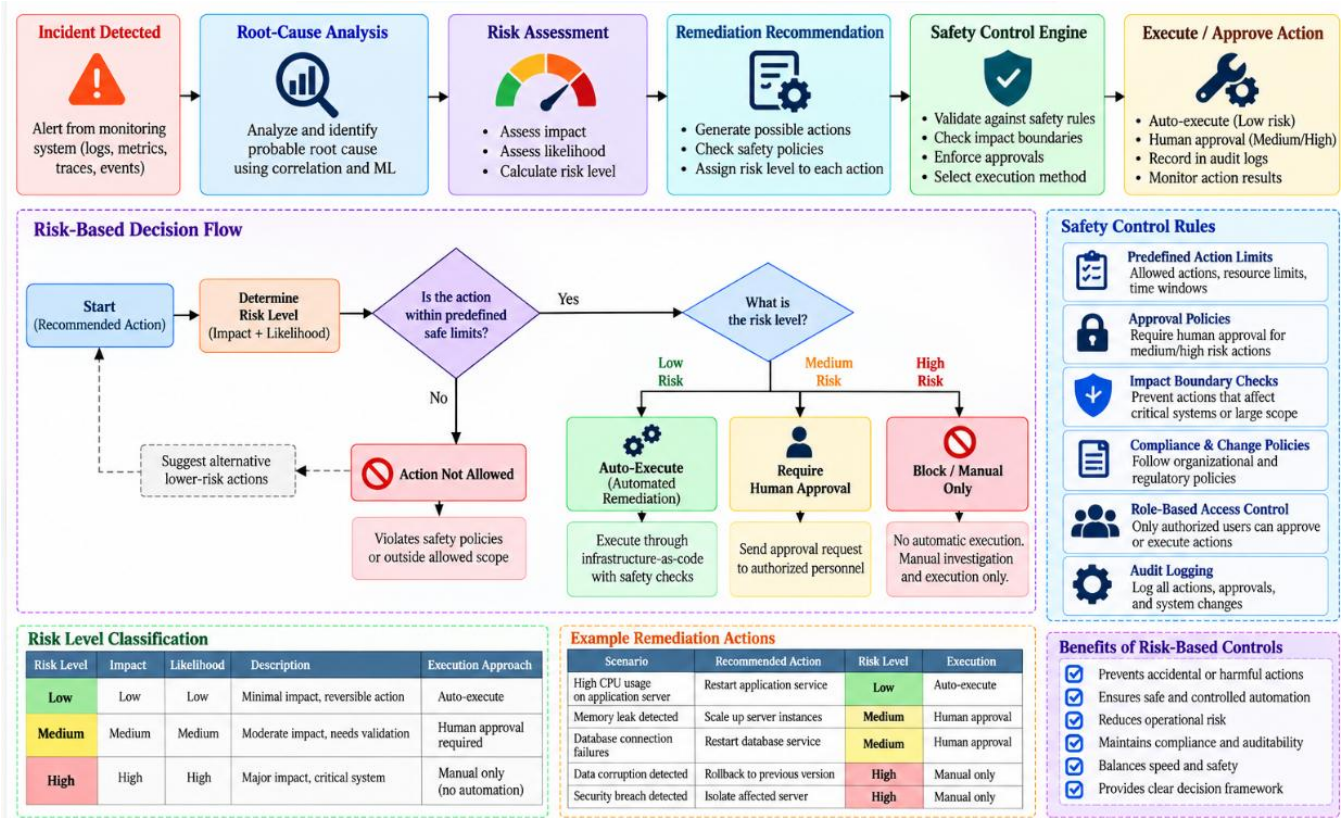


Figure 10. Risk matrix for autonomous remediation.

Risk	Example	Automation Policy
Low	Pre-approved routine recovery	May be automated within limits.
Medium	Rollback affecting a service	Validation and/or human approval.
High	Broad production configuration change	Human approval required.
Critical	Large blast radius or irreversible effect	Do not automate without explicit governance.

Safety Control Checklist

- Predefined allow-list of actions.
- Risk classification before execution.
- Service criticality and blast-radius checks.
- Role-based authorization.
- Human approval for high-impact actions.
- Dry-run or validation where possible.
- Rollback or recovery procedure.
- Complete audit trail.

Illustrative Incident Dashboard



Figure 11. Illustrative dashboard design derived from the case requirements.

UI Area	Information	Benefit
Incident Header	ID, severity, status, affected service	Situational awareness
Timeline	Detection, correlation, diagnosis, approval, action	Incident progression
Probable Cause	Ranked cause and evidence	Faster diagnosis
Recommended Action	Action, risk, approval state	Controlled decision-making
Audit Trail	Actor, time, action, result	Accountability

Testing Strategy and Sample Test Cases

Testing Level	Focus	Example
Unit Testing	Individual algorithms	Alert grouping logic
Integration Testing	Service interactions	Monitoring → ingestion → correlation
System Testing	End-to-end workflow	Alert → RCA → approved remediation
Security Testing	Authorization	Unauthorized approval rejection
Performance Testing	Volume / latency	High-volume alert burst
Failure Testing	Resilience	RCA service unavailable
UAT	Operational usability	SRE validates incident workflow

Sample Test Cases

ID	Scenario	Expected Result
TC-01	Duplicate alerts	Related alerts are grouped.
TC-02	Deployment followed by errors	Deployment appears as relevant evidence.
TC-03	Multiple causes	Candidates are ranked with evidence.
TC-04	Low-risk approved action	Action executes and is audited.
TC-05	High-impact action	System requests human approval.
TC-06	Unauthorized approval	Request is rejected and audited.
TC-07	Malformed event	Event is rejected/flagged safely.
TC-08	RCA unavailable	Degraded state is reported and data preserved.

Evaluation Metrics, Roadmap and Maintenance

Metric	Definition	Purpose
MTTD	Time to detect incident	Detection effectiveness
MTTI	Time to probable cause	Diagnostic efficiency
MTTR	Time to recovery	Recovery effectiveness
Alert Reduction Rate	Reduction in redundant alerts	Alert-overload reduction
Recommendation Acceptance	Recommendations accepted by responders	Usefulness / trust
Automation Success Rate	Successful automated actions	Safety and quality
Audit Completeness	Required actions with records	Governance quality

Suggested Development Roadmap

Phase	Deliverables
1 – Foundation	Data model, authentication, ingestion, dashboard
2 – Correlation	Normalization, alert grouping, incident timeline
3 – Analysis	Dependency context and root-cause ranking
4 – Governance	Policy, risk classification, approval workflow
5 – Controlled Automation	Low-risk remediation and rollback
6 – Optimization	Historical learning and predictive analysis

Maintenance Plan

- Review correlation rules after architecture or deployment changes.
- Monitor connector health and data quality.
- Review automation allow-lists and risk policies periodically.
- Audit access permissions and approval roles.
- Track MTTD, MTTI and MTTR for continuous improvement.
- Update tests when new integrations or actions are introduced.

Detailed Problem Decomposition

The assigned problem can be decomposed into connected engineering problems. This decomposition helps define system boundaries, responsibilities, measurable outcomes, and implementation priorities without changing the original case-study requirements.

Problem Area	Detailed Issue	Required Capability	Expected Benefit
Observability fragmentation	Signals are distributed across monitoring and logging systems.	Unified ingestion and normalization.	Single operational context.
Alert overload	Many alerts may represent one underlying incident.	Alert grouping and deduplication.	Reduced responder workload.
Root-cause complexity	Symptoms may appear across dependent services.	Dependency-aware correlation.	Faster diagnosis.
Deployment correlation	Recent changes may be related to failures.	Deployment-event integration.	Evidence-based diagnosis.
Remediation risk	Automatic actions can cause additional impact.	Risk classification and approval.	Safer automation.
Knowledge reuse	Past incidents contain useful evidence.	Incident history and audit store.	Continuous operational learning.
Governance	High-impact changes need accountability.	Approval workflow and audit trail.	Controlled response.

Assumptions and Constraints

- The platform depends on access to operational data from supported monitoring, logging, incident, deployment, and infrastructure systems.
- Data from different sources may use different formats and timestamps, so normalization is necessary.
- Automation is constrained by predefined safety boundaries rather than unrestricted autonomous decision-making.

- High-impact changes remain subject to human approval.
- The usefulness of root-cause analysis depends on the quality, completeness, and timeliness of available evidence.
- The platform should complement existing operational tools rather than unnecessarily replacing every existing system.

Success Conditions

- Responders can obtain a consolidated incident context quickly.
- Related alerts are grouped with fewer duplicate investigations.
- Probable causes are presented with understandable supporting evidence.
- Remediation recommendations are traceable to the incident context.
- Automated actions remain within approved risk boundaries.
- Every significant decision and action can be reviewed later.

Requirements Traceability Matrix

The following traceability matrix connects the problem statement, proposed requirements, design modules, and measurable verification. This provides a clear link between what the assessment asks the platform to achieve and how the proposed design addresses it.

Problem Requirement	System Requirement	Design Module	Verification / Evidence
Correlate monitoring data	System shall ingest and correlate operational events.	Ingestion + Correlation	Integration and system tests.
Correlate deployment changes	Deployment events shall be available in incident context.	Deployment Connector	Deployment-correlation test.
Use service dependencies	System shall maintain dependency context.	Dependency Module	Dependency impact test.
Identify probable causes	System shall rank candidate causes.	RCA Module	RCA scenario tests.
Recommend remediation	System shall generate action recommendations.	Remediation Module	Recommendation review.
Automate low-risk recovery	Approved low-risk actions may execute automatically.	Automation Module	Controlled execution test.
Human approval for high-impact changes	High-impact actions shall stop at approval gate.	Approval Module	Authorization test.
Maintain incident history	Incidents and outcomes shall be retained according to policy.	History Store	Audit/history verification.
Provide audit trail	Important decisions and actions shall be recorded.	Audit Module	Audit completeness test.

Priority Classification

Priority	Capability	Reason
P1 – Critical	Incident ingestion, correlation, RCA, approval controls	Required for the core problem.
P2 – High	Dependency mapping, remediation recommendations, dashboard	Improves diagnosis and operational usability.
P3 – Medium	Controlled low-risk automation, advanced reports	Reduces repetitive work after core safety is established.
P4 – Future	Predictive analysis and advanced historical learning	Optimization after reliable baseline operation.

Detailed Agile / Scrum Implementation Plan

Agile with Scrum practices is appropriate for this case because the proposed platform contains multiple integrations, evolving operational requirements, and safety-sensitive automation. The implementation should prioritize a reliable foundation before expanding automation.

Sprint	Focus	Major Deliverables	Review Outcome
Sprint 1	Foundation	Project structure, roles, requirements, data model	Stakeholder confirms scope.
Sprint 2	Ingestion	Monitoring and logging	Sample data successfully

		connectors	ingested.
Sprint 3	Normalization	Common event schema and validation	Data quality reviewed.
Sprint 4	Correlation	Alert grouping and incident timeline	Duplicate alerts reduced in test scenarios.
Sprint 5	RCA	Dependency context and cause ranking	Candidate causes shown with evidence.
Sprint 6	Governance	Risk policy and approval workflow	High-impact action is blocked until approval.
Sprint 7	Automation	Pre-approved low-risk remediation	Controlled action executes and is audited.
Sprint 8	Hardening	Security, performance, failure testing	Release-readiness assessment.

Definition of Done

- Requirement is implemented and reviewed.
- Unit and integration tests are completed for the affected component.
- Security and authorization behavior is verified.
- Logs and audit records are available where required.
- Documentation is updated.
- Demonstration is completed during sprint review.
- Known defects are recorded and prioritized.
- High-impact automation cannot bypass the approval policy.

Team Responsibilities

Role	Primary Responsibility
Product Owner	Prioritize operational value and acceptance criteria.
Scrum Master	Facilitate Scrum activities and remove process blockers.
Developers	Implement services, integrations, interfaces and tests.
DevOps / SRE	Validate operational behavior, reliability and deployment.
Security	Review access, authorization and automation safety.
Stakeholders	Provide feedback during reviews and validate usefulness.

Security, Privacy and Governance Analysis

Although the case is primarily an incident-response problem, the proposed platform has access to operational infrastructure information and potentially sensitive system data. Security and governance therefore need to be treated as core engineering requirements.

Security Area	Threat / Concern	Control
Authentication	Unauthorized users access the platform.	Strong authentication and session controls.
Authorization	User executes an action beyond their responsibility.	Role-based access and least privilege.
Secrets	Credentials for integrated tools are exposed.	Secure secret storage and controlled access.
Audit Integrity	Action history is modified or deleted.	Protected audit storage and access restrictions.
Automation Abuse	An attacker or faulty rule triggers recovery actions.	Allow-lists, risk checks and approval gates.
Data Exposure	Operational information is visible to unauthorized users.	Data classification and access control.
Connector Security	External integrations accept untrusted input.	Validation, authentication and secure interfaces.
Availability	The response platform becomes unavailable during an incident.	Redundancy, monitoring and recovery planning.

Governance Rules for Autonomous Actions

- Every automated action should have an owner and documented purpose.
- Actions should have a clearly defined risk level.
- Only approved action types should be executable.

- High-impact actions should require explicit authorization.
- Actions should be reversible where technically feasible.
- The platform should record the initiating incident, policy decision, actor or automation identity, timestamp, and outcome.
- Automation policies should be periodically reviewed using incident outcomes and operational feedback.

Operational Accountability

The platform should not be treated as an unrestricted autonomous operator. Its purpose is to reduce repetitive work while keeping safety and accountability visible. The responder should be able to understand why an action was recommended and whether the action is allowed by policy.

Comprehensive Conclusion and Final Assessment Summary

The Autonomous Incident Response and Root-Cause Analysis Platform addresses the central problem of fragmented operational information and slow incident diagnosis. The proposed solution brings monitoring data, logs, alerts, deployment changes, service dependencies, and incident history into a common analysis workflow.

The design emphasizes correlation before action. Instead of immediately executing a recovery command, the platform builds incident context, identifies probable causes, provides evidence, evaluates the risk of a proposed action, and then either executes an approved low-risk action or requests human approval for a high-impact change.

Major Contributions of the Proposed Design

- Provides a structured approach to combining heterogeneous operational signals.
- Reduces alert overload by grouping related events into meaningful incidents.
- Improves diagnosis through service dependency and deployment context.
- Uses evidence-based probable-cause ranking rather than unexplained automation.
- Introduces explicit risk boundaries for autonomous remediation.
- Preserves human approval for high-impact operational changes.
- Creates an audit trail that supports accountability and future incident learning.
- Supports Agile/Scrum development so the platform can evolve incrementally.
- Includes testing, security, governance, measurement and maintenance considerations.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria Mark

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25